

DEVOXX BELGIUM 2026

Building LLM inference engines in Java

with TornadoVM and the NVIDIA ecosystem

Michalis Papadimitriou

Neo4j

University of Manchester

Orion Papadakis

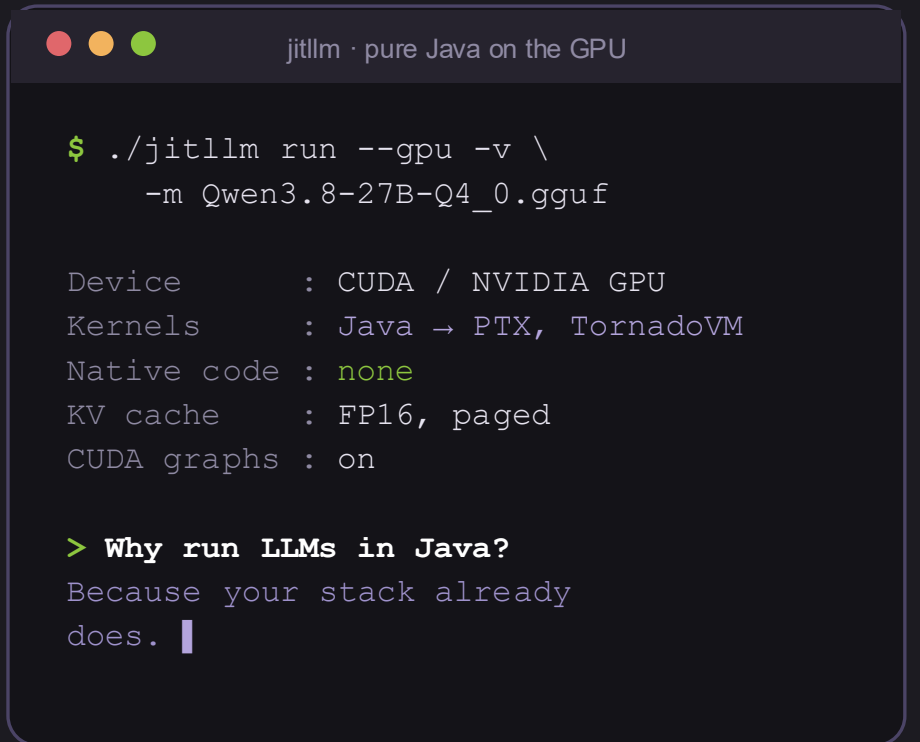
TornadoVM

Beehive Lab · The University of Manchester

DEVOXX™
BELGIUM

Antwerp
session date · room

tornadovm.org



```
jittllm · pure Java on the GPU

$ ./jittllm run --gpu -v \
  -m Qwen3.8-27B-Q4_0.gguf

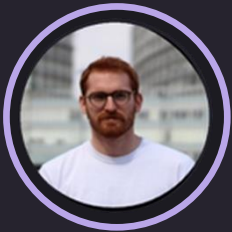
Device      : CUDA / NVIDIA GPU
Kernels     : Java → PTX, TornadoVM
Native code : none
KV cache    : FP16, paged
CUDA graphs : on

> Why run LLMs in Java?
Because your stack already
does. |
```

0.94× of llama.cpp decode on Qwen3.8-27B, from Java

j ⚡ tLLM

About us



Michalis Papadimitriou

Senior Software Engineer · Neo4j
Research Fellow · University of Manchester

TornadoVM Contributor

PREVIOUSLY

ML Compiler Engineer · OctoAI (acquired by NVIDIA)
Research Software Engineer · Huawei Research Labs

 @mikepapadim
 @mikepapadim
 @mikepapadim.bsky.social

michail.papadimitriou@manchester.ac.uk



Orion Papadakis

Research Associate · University of Manchester

TornadoVM Contributor

 @orionpapadakis
 @ orionpapadakis
 @ orionpapadakis.bsky.social

orion.papadakis@manchester.ac.uk



GPULlama3.java is now jitLLM.

Java Inference Tornado toolkit for LLMs that
goes beyond Llama3

github.com/beehive-lab/jitllm

j ⚡ tLLM

INTRO Demo: **What we can run today?**

Java writes Java for the GPU, on the GPU.



A Java LLM engine powered by TornadoVM generates Java code that runs on the same GPU through TornadoVM

The AI runs in Java

jitLLM does LLM inference with Java kernels on the GPU. No Python, no C++.

The AI writes Java

The model writes a plain Java method with `@Parallel` loops. No CUDA.

Java becomes CUDA

TornadoVM JIT-compiles that method and runs it on the GPU that just ran the model.

***From prompt to running
GPU kernel without leaving
the JVM.***

But why Java? Java is slow...

Pure Java, at llama.cpp speed on GPUs

Same GPU, same GGUF weights. Every kernel written in Java and compiled by TornadoVM at runtime.

0.94×

DECODE · 27B

of llama.cpp decode on Qwen3.8-27B

40.5 vs 43.2 tok/s · Q4_0 · pure JIT, zero native code

1.12×

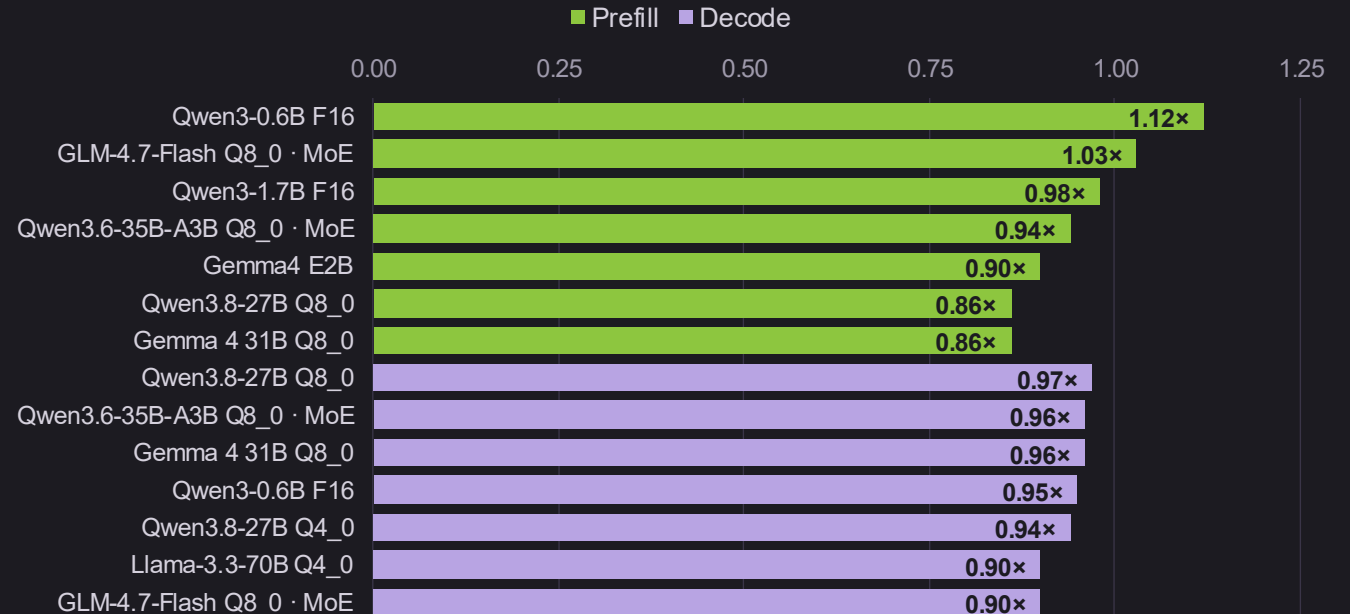
PREFILL · HYBRID

faster than llama.cpp on prefill

Qwen3-0.6B F16 pp512 · 50,300 vs 44,805 tok/s · cuBLAS + Java

No C++, no CUDA C, no JNI.

Throughput relative to llama.cpp = 1.00



14 configurations at ≥85% of llama.cpp · prefill vs llama.cpp ubatch 512 · GPU: fill in

What we will cover

- 01 Why the JVM, and TornadoVM in five minutes** from a Java loop to a CUDA kernel
- 02 Anatomy of a Java inference engine** jittlm's architecture ·
- 03 JitLLM vs Llamacpp** live benchmarking - performance
- 04 The NVIDIA ecosystem from Java** cuBLAS, cuDNN, CUDA graphs, tensor cores ·
- 05 Quarkus, LangChain4j, Spring Boot**
- 06 Summary and Future Roadmap**

01

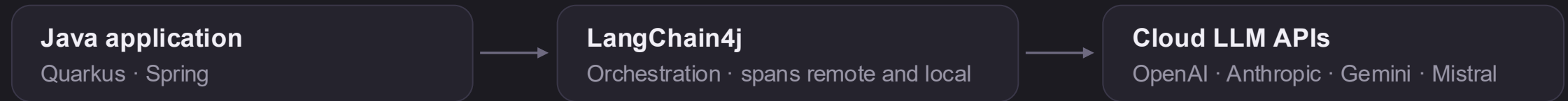
Why the JVM and TornadoVM

Local inference without a Python sidecar, and the framework that makes Java kernels run on GPUs.

Java developers in the LLM era

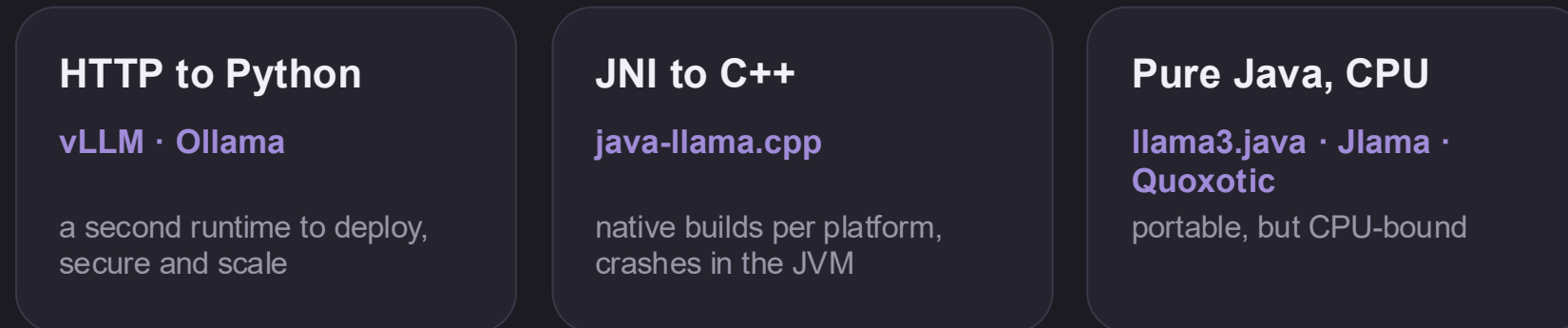
Java's legacy in the AI era calls for Java-native inference

REMOTE MODELS



LOCAL MODELS · ON PREMISE

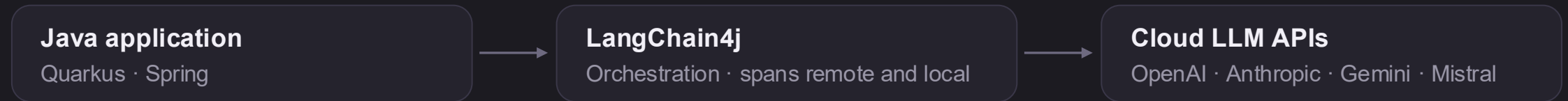
privacy · security · custom weights · offline · domain-specific



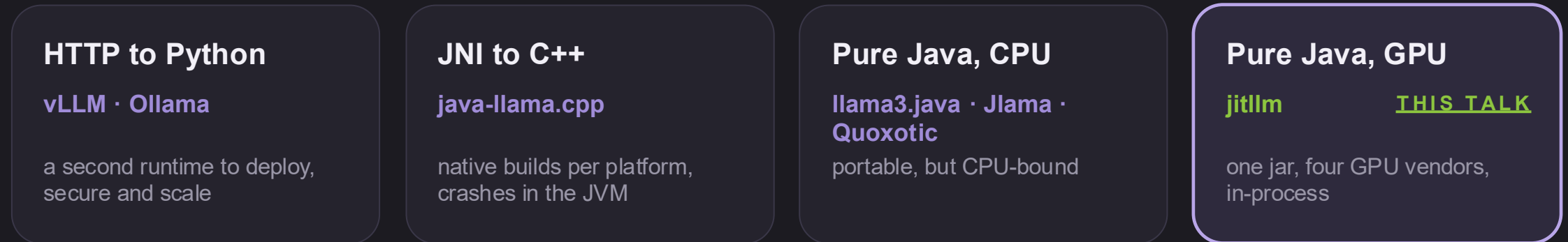
Java developers in the LLM era

Java's legacy in the AI era calls for Java-native inference

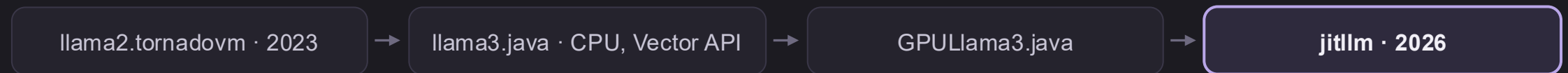
REMOTE MODELS



LOCAL MODELS · ON PREMISE privacy · security · custom weights · offline · domain-specific



Timeline of Java + Local Inference on GPUs



The TornadoVM ecosystem

Any JDK from 21 to 27, three GPU backends, and the tools and integrations around them.

JDK 21–27 DISTRIBUTIONS

OpenJDK	Microsoft JDK
GraalVM	Azul Zulu
Eclipse Temurin	SapMachine
Red Hat Mandrel	

runs on any JDK



compiles per GPU

HARDWARE BACKENDS



OpenCL

AMD · Intel · NVIDIA



CUDA

NVIDIA



Metal

Apple Silicon

DEVELOPER TOOLS

-  TornadoVMPulse
-  TornadoViz
-  TornadoInsight
-  NVIDIA Nsight Systems

DOCKER IMAGES



NVIDIA and Intel images

DISTRIBUTION



Maven Central · SDKMAN!

INTEGRATIONS



Spring Boot



Quarkus



LangChain4j



Flink

Active since 2018 · 30+ contributors · 30+ research papers · used by the European Space Agency (ESA)

TornadoVM in a nutshell (not a VM, just a library)

Runs on the JDK you already have. Not a JVM, not a binding layer.

Your Java application

ordinary bytecode, still running on the JVM

Tornado API

@Parallel · KernelContext · TileContext · TaskGraph · TornadoExecutionPlan

Runtime

task graph · transfer tracking · device memory · CUDA graphs · streams · NVTX

JIT compiler

bytecode → Tornado IR → CUDA C (NVRTC) or CUDA Tile C++ (tileiras) → cubin

CUDA backend

SIMT + CUDA Tile

OpenCL

AMD · Intel · NVIDIA

Metal

Apple Silicon

Library providers

cuBLAS · cuFFT · cuDNN · ...

Four ways to program the GPU

From least to most control, plus the vendor's own kernels. Mix them freely in one graph.

01

@Parallel

Annotate a loop. The compiler maps iterations to GPU threads.

easiest start

02

KernelContext

The CUDA thread model in Java: ids, shared memory, barriers, warp shuffles, mma.sync.

full control

03

TileContext

Describe one tile block. The CUDA Tile compiler picks threads and tensor-core instructions.

NEW · CUDA Tile

04

libraryTask

NVIDIA's optimised kernel as a node in the same graph, on the same buffers.

cuBLAS · cuFFT · cuDNN

Four ways to program the GPU

More in our talk on TornadoVM internals!

From least to most control, plus the vendor's own kernels. Mix them freely in one graph.

01 **easiest start**

@Parallel

Annotate a loop; iterations become GPU threads.

```
void add(FloatArray a,
        FloatArray b,
        FloatArray c) {
    for (@Parallel int i = 0;
        i < c.getSize(); i++)
        c.set(i, a.get(i)
            + b.get(i));
}
```

02 **full control**

KernelContext

The CUDA thread model: ids, shared memory, barriers.

```
void sum(KernelContext ctx,
        FloatArray a,
        FloatArray out) {
    int t = ctx.localIdx;
    float[] s = ctx
        .allocateFloatLocalArray(256);
    s[t] = a.get(ctx.globalIdx);
    ctx.localBarrier();
    for (int h=128; h>0; h/=2) {
        if (t < h) s[t] += s[t + h];
        ctx.localBarrier();
    }
    if (t == 0)
        out.set(ctx.groupIdx, s[0]);
}
```

03 **NEW · CUDA Tile**

TileContext

Describe one tile block; the tile compiler does the rest.

```
void mm(TileContext tc, ...) {
    var av = tc.partition(
        tc.view(a, m, k), TM, TK);
    var bv = tc.partition(
        tc.view(b, k, n), TK, TN);
    int r = tc.bidX();
    int c = tc.bidY();
    Tile acc=tc.zeros(F32,TM,TN);
    for (int s=0; s<k/TK; s++)
        acc = tc.mma(av.load(r,s),
            bv.load(s,c), acc);
    cv.store(acc, r, c);
}
```

04 **cuBLAS · cuFFT · cuDNN**

libraryTask

NVIDIA's kernel as a node, on the same buffers.

```
new TaskGraph("g")
    .transferToDevice(
        EVERY_EXECUTION, m, v)
    .task("scale",
        App::scale, m)
    .libraryTask("sgemv",
        CuBlas::cublasSgemv,
        OP_T, rows, cols, 1f,
        m, lda, v, 1, 0f, out, 1)
    .task("bias",
        App::bias, out)
    .transferToHost(
        EVERY_EXECUTION, out);
```

Anatomy of a Java GPU inference engine

Five abstractions that turn a model port into an engine, and the API that hides them.

Bring vLLM's architecture, inside the JVM for GPUs

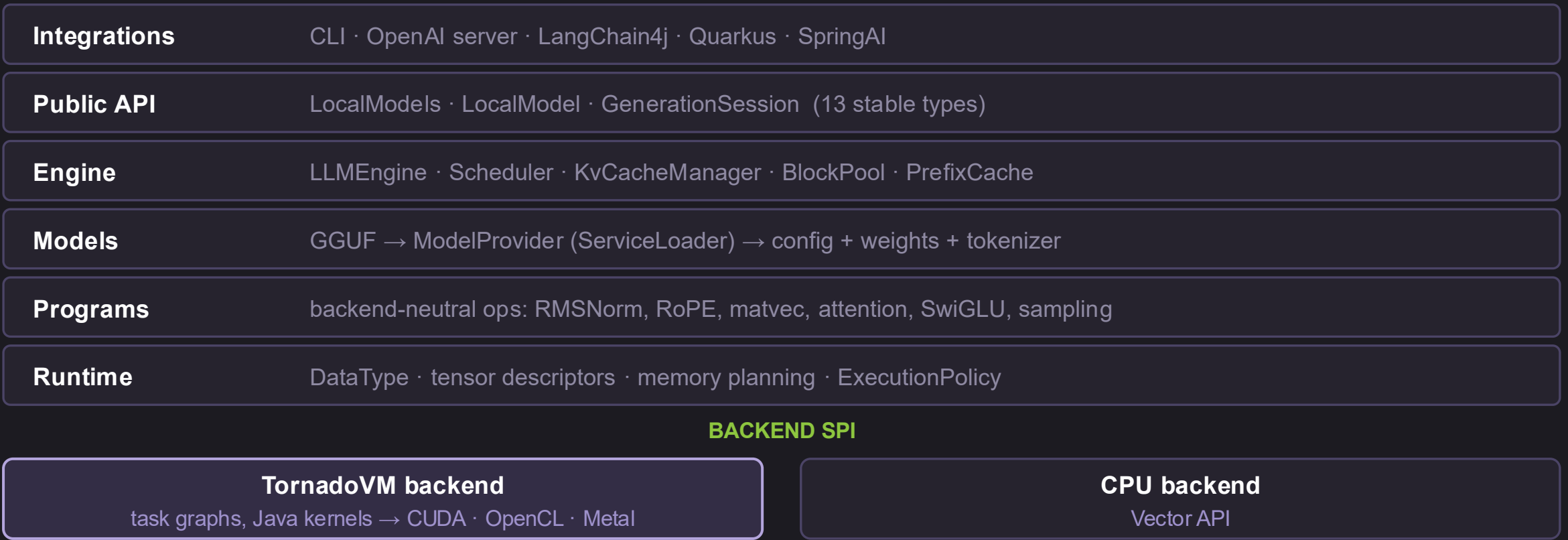
What leading frameworks can do, but in plain modern Java!

	jitllm	vLLM	llama.cpp
Runtime	Java 21+ JIT to GPU	Python + CUDA/Triton	C/C++
GPU vendors	CUDA · OpenCL · Metal, one jar	CUDA, ROCm, Intel, TPU	CUDA, HIP, SYCL, Vulkan, Metal
Model format	GGUF	safetensors (GGUF exp.)	GGUF
Paged KV cache	yes · block pool + table	yes · PagedAttention	yes · unified KV cells
Continuous batching	exp. · CUDA, FP16, greedy	yes	yes · --parallel
Tensor-core prefill	yes · Java <code>mma.sync</code>	yes	yes
Vendor libraries	exp. · cuBLAS + cuDNN tasks	yes	yes · cuBLAS
Speculative decoding	exp. · Qwen3.5 MTP, CPU	yes · EAGLE, MTP, draft	yes · draft model
Structured output	no	yes · xgrammar	yes · GBNF
Multi-GPU	no · roadmap	yes · TP/PP/EP/DP	yes · layer/row split
Embed in a Java app	yes · in-process library	HTTP only	JNI bindings

JitLLM, layer by layer

TornadoVM is hidden on bottom. On top ONLY modern Java.

depends on

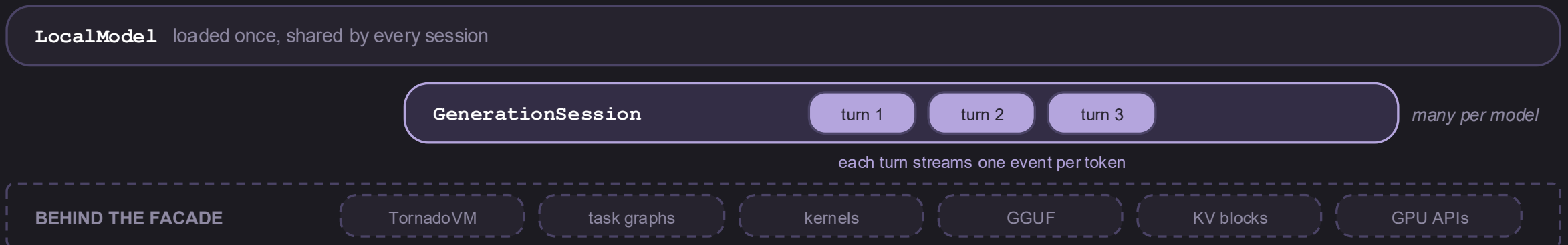


Public API

Four calls, two lifetimes. Nothing from TornadoVM, GGUF or the GPU leaks through.

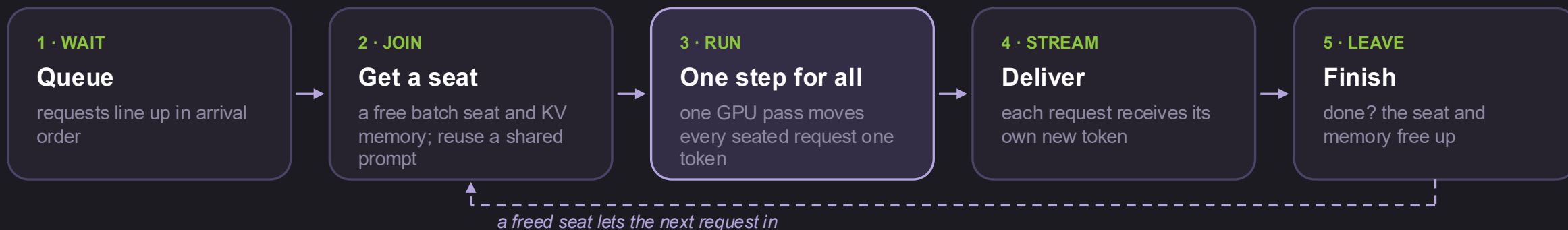


LIFETIMES

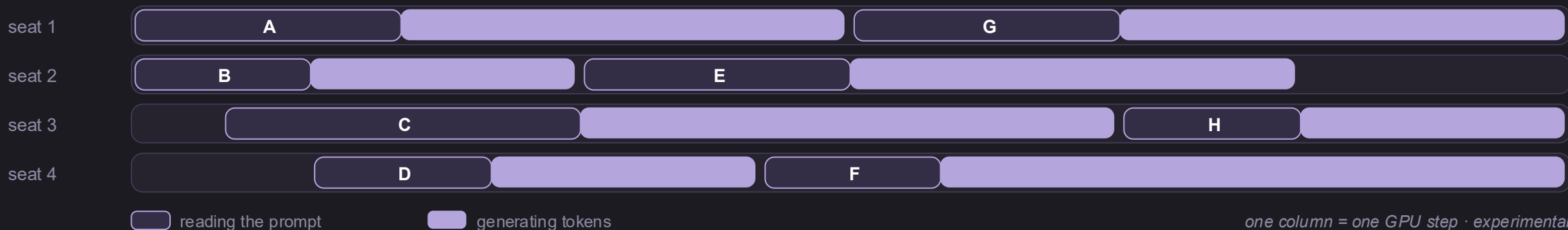


Engine: the orchestrator

Drives the requests: many conversations share one GPU, one batch and one KV pool.



CONTINUOUS BATCHING requests join and leave between steps; the GPU never waits for the slowest one



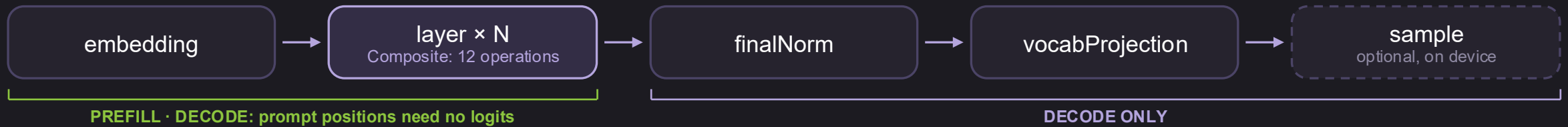
Models: Abstractify multi-axis configs

A Model is one pick per axis. Each axis belongs to one abstraction.

FAMILY		WEIGHTS	MODE	KV CACHE	DEVICE	FEATURES
ModelProvider · ModelArchitecture		supportedDataTypes ()	ExecutionPolicy	KvStorageFactory	DeviceResolver	opt-in
Llama 3 1B · 3B · 8B	Qwen3 0.6B → 8B	F32	standard	FP16	CPU	CUDA graphs
Qwen3.5 / 3.8 27B	Mistral 7B v0.3	F16 · BF16	sequential	FP32	CUDA	device sampling
Devstral Small	Qwen2 / 2.5 0.5B · 1.5B	Q8_0	batched (MMA)		OpenCL	cuBLAS / cuDNN
DeepSeek-R1-Distill Qwen 1.5B	Phi-3 mini	Q4_0			Metal	prefix cache
Granite 3.x / 4.0 2B · 1B	Gemma 4 E2B	Q4_1 · Q4_K				
Qwen2-MoE fork branch		Q5_K · Q6_K				

Program: A combo of building blocks

A program describes what to compute, from one shared vocabulary. Never how, never on which device.



A FAMILY IS A PICK five families, 14 operations, cross-family reusable blocks

	Embed	Rms Norm	Mat Vec	RoPE	KV append	Attention	Swi GLU	Resid. add	Vocab proj.	Arg Max	Bias Add	Split QKV	Split gate-up	Scale
Llama	●	●	●	●	●	●	●	●	●	●	○	○	○	○
Qwen3	●	●	●	●	●	●	●	●	●	●	○	○	○	○
Qwen2	●	●	●	●	●	●	●	●	●	●	●	○	○	○
Phi-3	●	●	●	●	●	●	●	●	●	●	○	●	●	○
Granite	●	●	●	●	●	●	●	●	●	●	○	○	○	●

shared core: every family

one family each

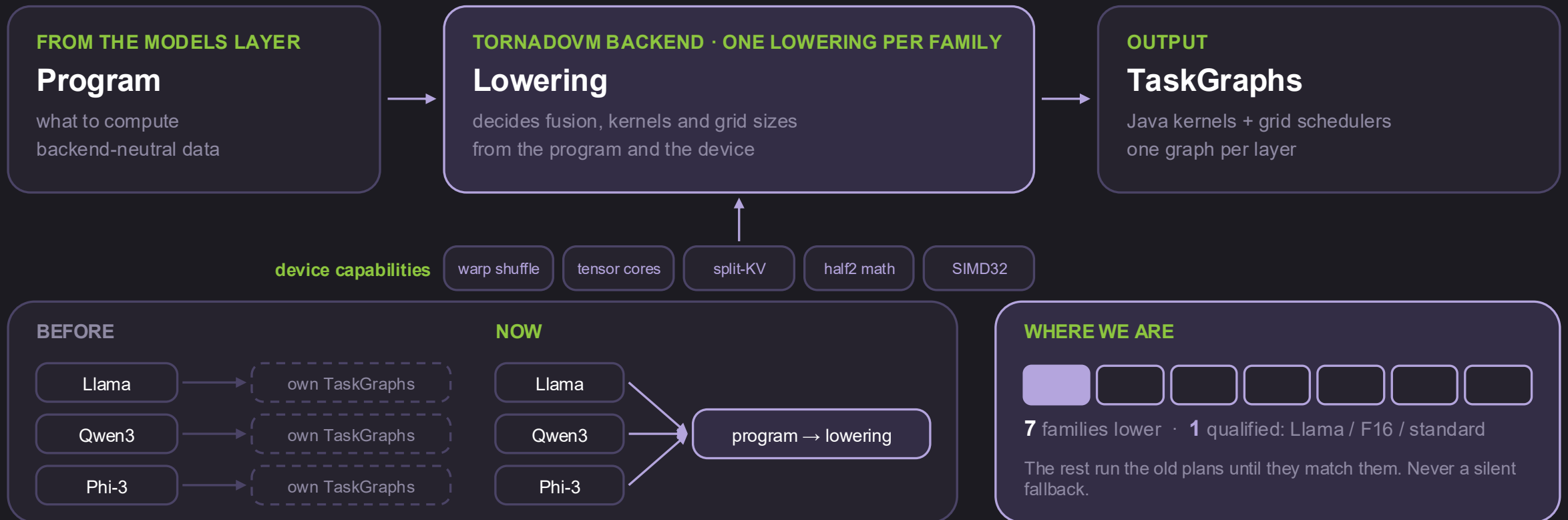
SEALED VOCABULARY · 26

12 more operations are defined and run on the CPU, waiting for their families:

MoeRouter	ExpertFFN	WeightedAcc
GeGLU	LogitSoftCap	CausalConv1d
DeltaRule	GatedNorm	L2Norm
MatMul	Softmax	Sample

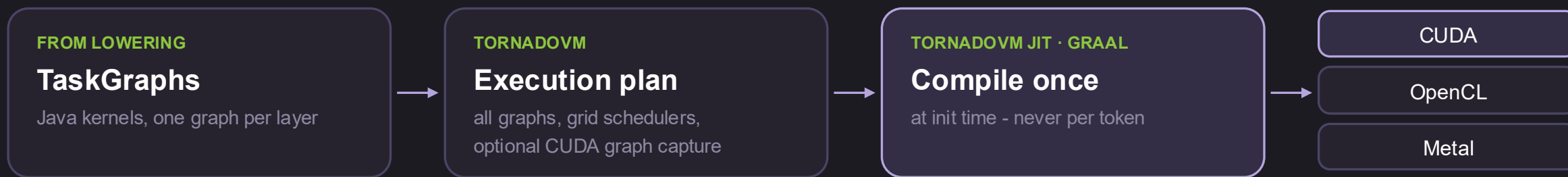
Program Lowering

From a backend-neutral ops into TornadoVM TaskGraphs.

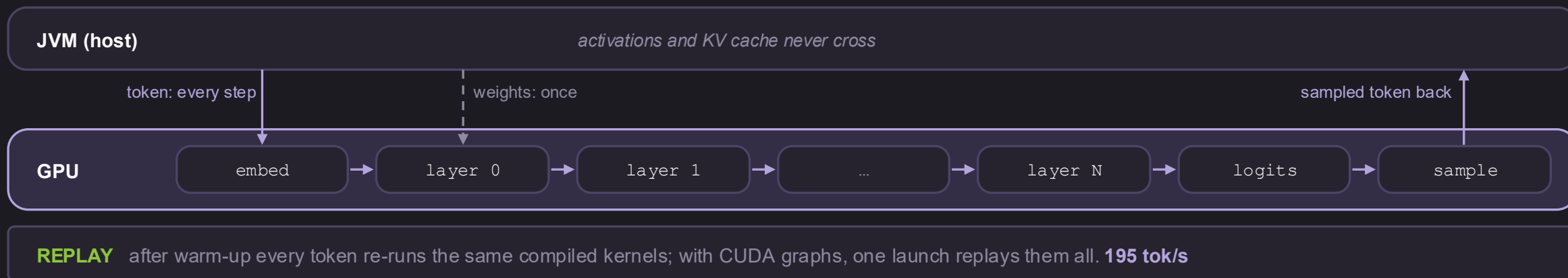


TornadoVM Backend: all the way down to JIT

It hands the TaskGraphs to TornadoVM, which JIT-compiles them into device code, once.



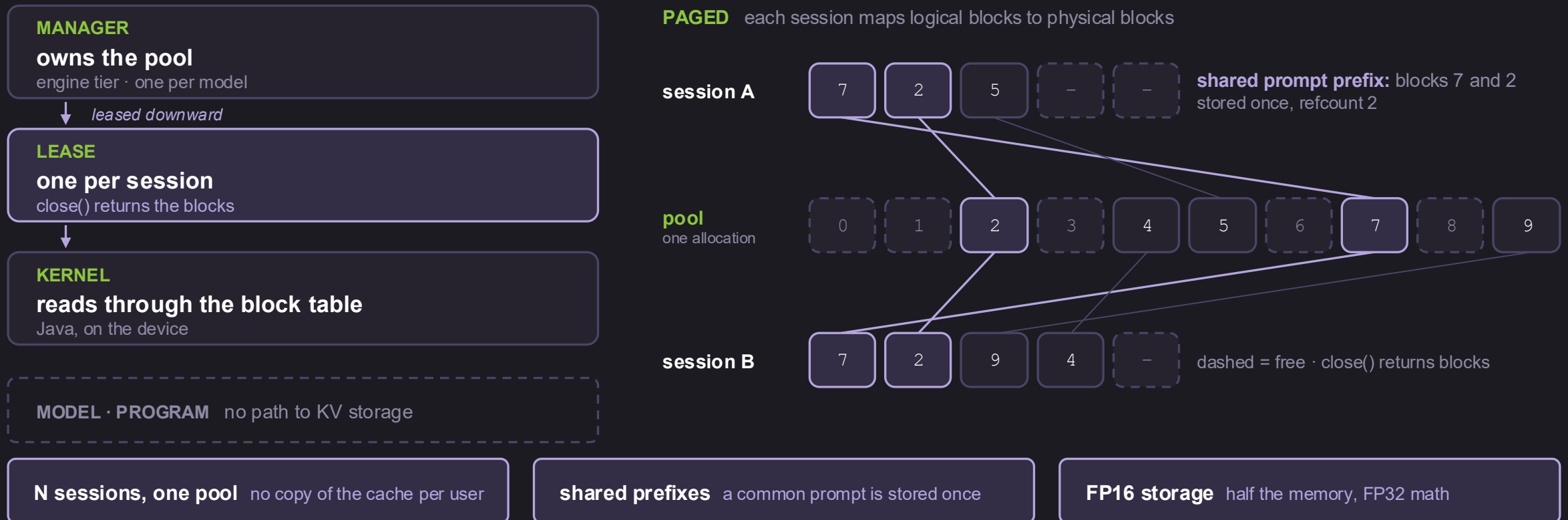
DATA STAYS ON THE GPU the JVM sends a token in and gets a token back



Paged KV Cache: Pool of blocks

SCALABILITY

One pool serves many sessions. A manager leases paged blocks; no model or program can reach them.



DEMO 2

jitllm vs llama.cpp

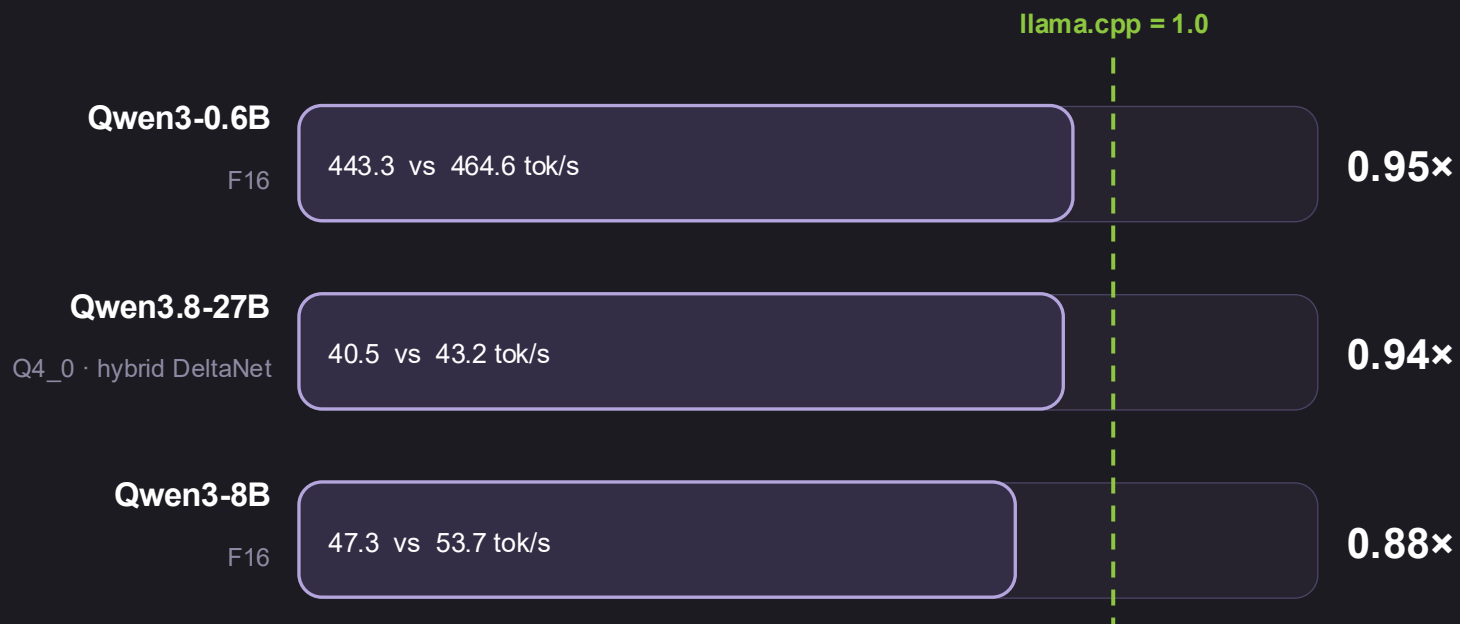
Same GGUF, same GPU, measured the same way:
jitllm bench matches llama-bench's pp/tg matrix on
purpose.

Decode: parity, in pure Java

tg128 tok/s · same GPU, same GGUF · every kernel Java, JIT-compiled by TornadoVM, no native libraries

0.95×

of llama.cpp decode speed, best case,
with no C++ and no CUDA C

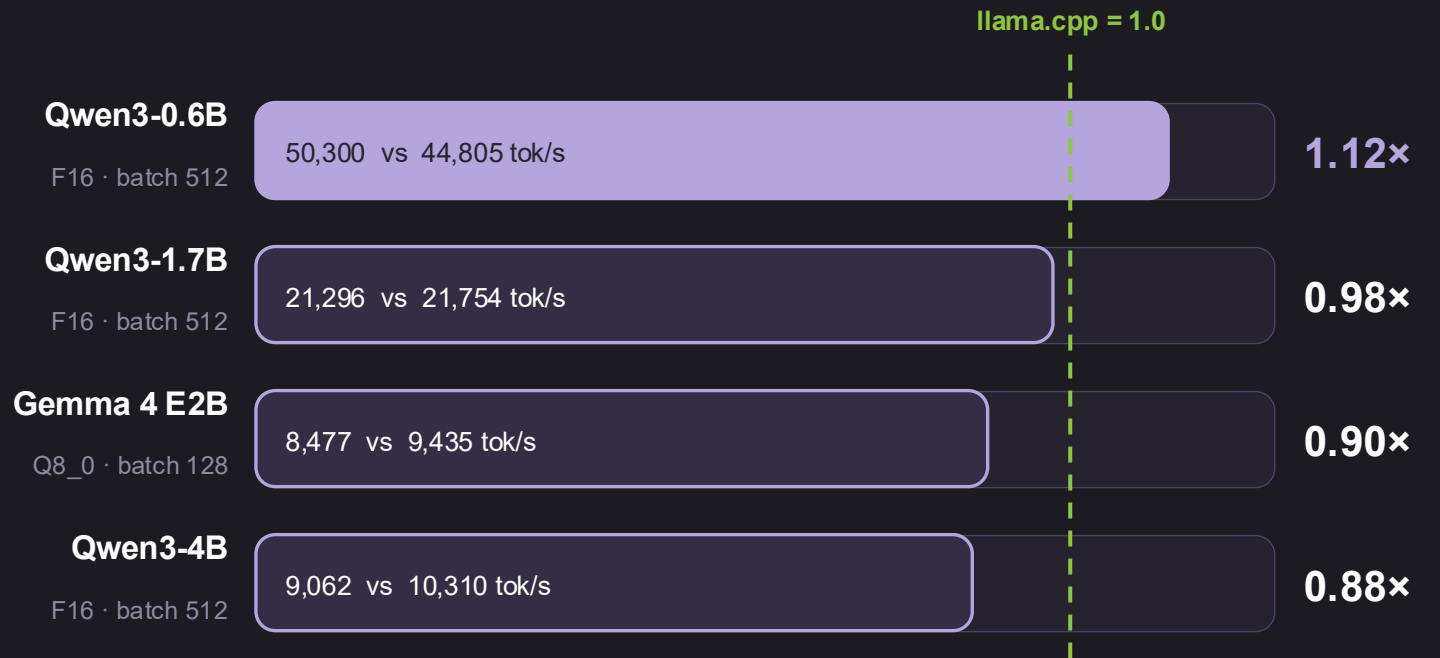


Prefill: parity on short prompts

pp512 tok/s · same GPU, same GGUF · jitllm batch = llama.cpp ubatch · Java kernels + cuBLAS

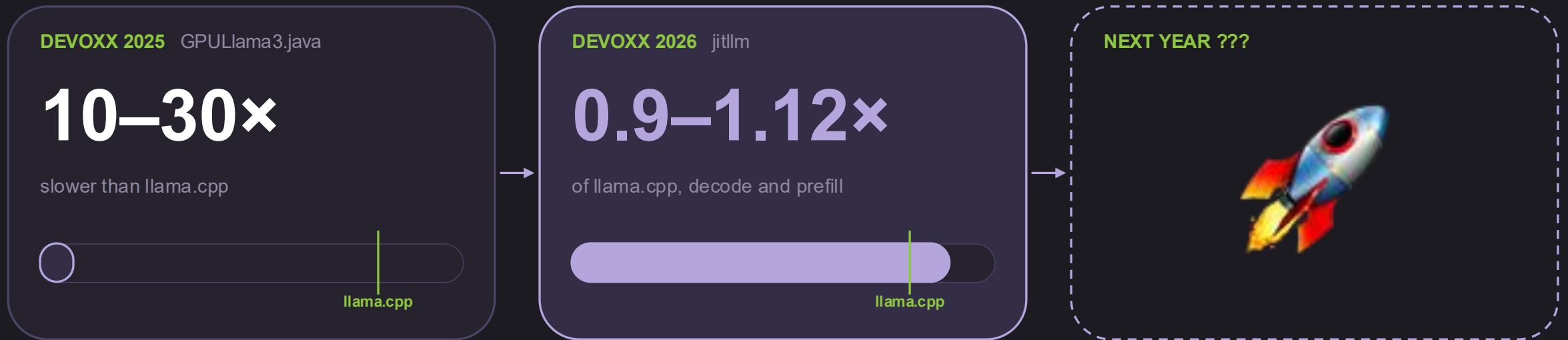
1.12×

faster than llama.cpp on Qwen3-0.6B
prefill, at the same batch size



From 10–30× slower to parity, in one year

Same idea, every kernel in Java: GPULlama3.java at Devoxx 2025, JitLLM today.



WHAT CHANGED IN A YEAR

tensor-core prefill in Java

fused kernels · warp matvec

CUDA graphs

cuBLAS from Java

quantized weights stay on GPU

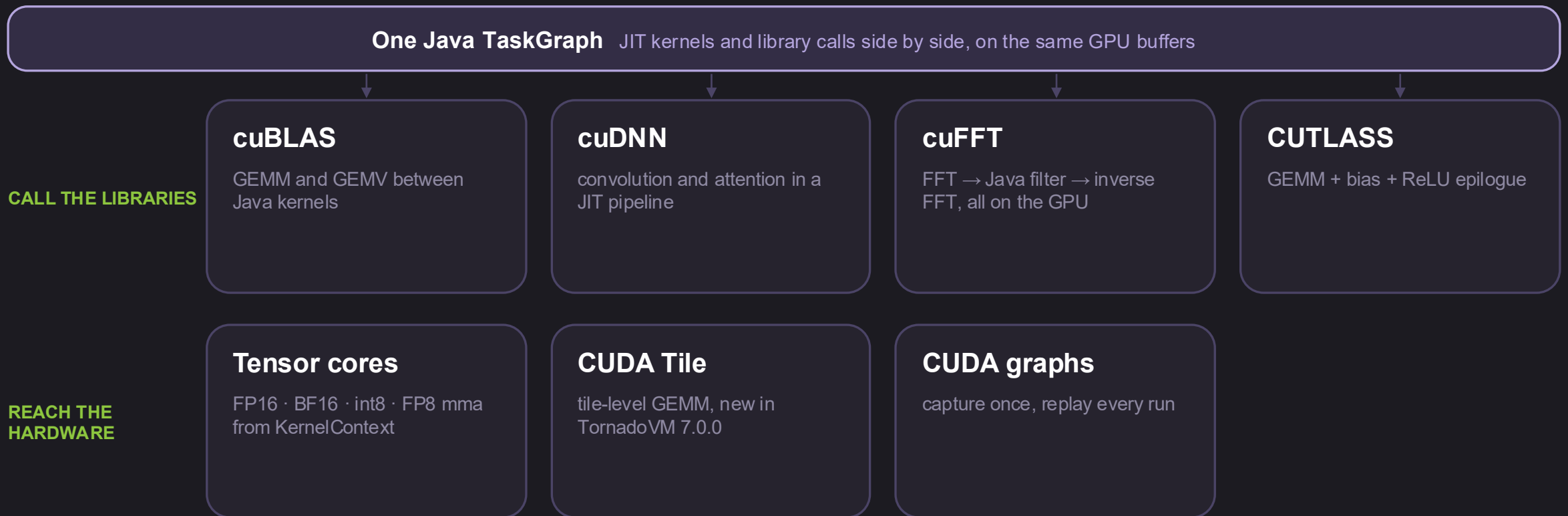
paged FP16 KV cache

Accessing the NVIDIA ecosystem from Java

Java kernels and NVIDIA libraries in one TaskGraph,
on shared GPU buffers. No JNI, no C++.

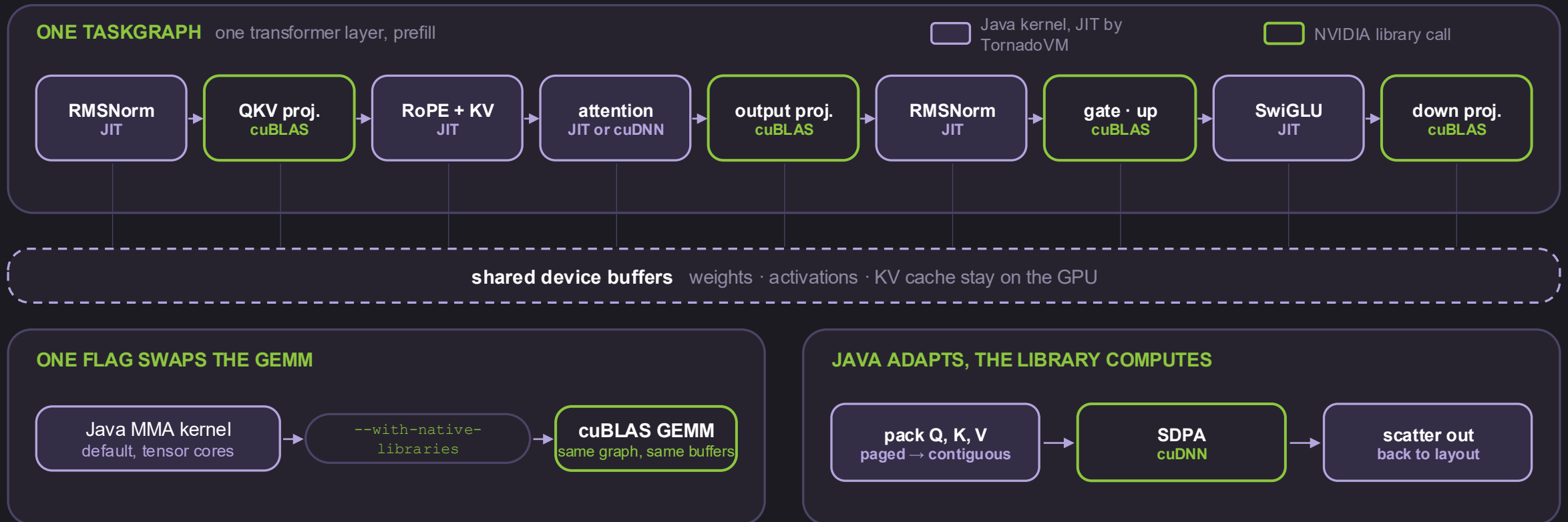
The NVIDIA ecosystem, available from Java

Multiple pieces of the CUDA ecosystem, each one accessible from Java.



JIT kernels mixed with NVIDIA libraries

JIT adapts on custom compute, the library does the heavy math.



What actually runs

With `--print-taskgraph-chain` · Qwen3-0.6B F16 · `--with-native-libraries`

Prefill calls libraries, decode stays Java

Prefill

COMPUTE-BOUND

whole prompt at once: matrix × matrix, a library job



Decode

BANDWIDTH-BOUND

one token: matrix × vector, Java already reads at memory speed



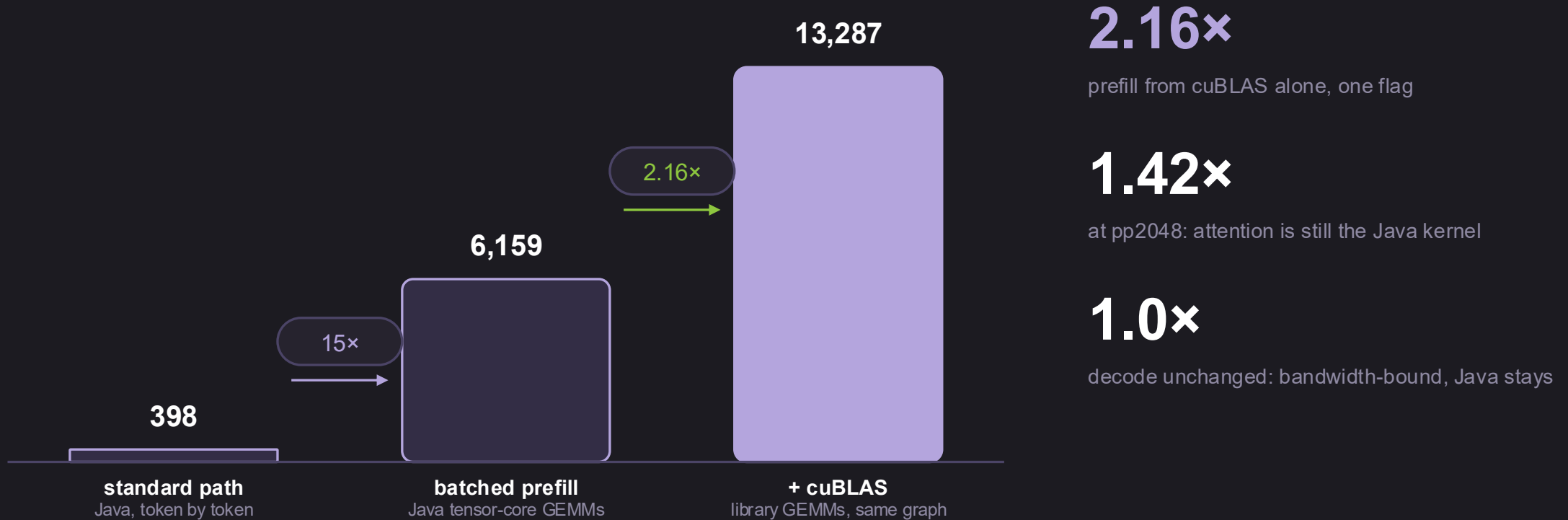
 Java kernel, JIT by TornadoVM

 NVIDIA library call

7 prefill graphs × 4 layers · 21 decode graphs · simplified

One flag, twice the prefill

Qwen3-0.6B F16 · pp512 prefill tok/s · batch 128 · RTX 4090 · TornadoVM 7.1.0 · 3 reps
./jitllm --with-native-libraries



Quarkus, LangChain4j and Spring Boot

Three ways in: from a URL change to the engine
inside your own JVM.

j ⚡ tLLM



Three ways into your app

Same model, same GPU. Choose how close to your code the engine runs.

IN YOUR JVM · no extra process

OVER HTTP · any language

1 Framework integration

Quarkus · LangChain4j · Spring AI



```
@RegisterAiService
interface Assistant {
    String chat(String q);
}
```

STATUS works today

2 Embed the Java API

any JVM app · a Spring @Bean



```
try (var model =
    LocalModels.load(path)) {
    // sessions, generate
}
```

STATUS works today

3 OpenAI-compatible server

Quarkus · Spring AI · curl · Python



```
$ jitllm serve -m model.gguf --gpu

base-url=
http://localhost:8089/v1
```

STATUS live today, deterministic

DEMO 4

Quarkus + LangChain4j

JitLLM serve in one pane, a Quarkus AI service and a LangChain4j chat with memory and streaming in the other.

“

Every layer exists!

**JitLLM proves Java
libraries can be built
and run on GPUs.**



Thank you! Questions?

Resources to take home.

- 1 **jitllm**
github.com/beehive-lab/jitllm
- 2 **TornadoVM**
github.com/beehive-lab/TornadoVM
- 3 **Maven Central**
io.github.beehive-lab:jitllm
- 4 **LangChain4j provider**
docs.langchain4j.dev · langchain4j-jitllm
- 5 **TornadoVM CUDA demos**
github.com/mikepapadim/tornadovm-devoxx2026-cuda-demos
- 6 **SDKMAN!**
sdkman.io/sdks/tornadovm

Funded in part by the EU projects AERO, P2CODE, ENCRYPT and TANGO.

Scan for jitllm



github.com/beehive-lab/jitllm

GET IN TOUCH

Michalis Papadimitriou

Neo4j · @mikepapadim

Orion Papadakis

TornadoVM ·
@orionpapadakis

We value your feedback!

Rate this talk



DEVOXX™

Backup

03

Making it fast

bandwidth vs compute

Decode is bandwidth, prefill is compute, and quantization is both.

Two phases, two bottlenecks

Every optimization in this talk targets one of these.

Prefill

the whole prompt at once

- All prompt tokens known up front
- Batched: matrix $\times$ matrix
- Compute-bound: tensor cores, cuBLAS
- Drives time to first token

Decode

one token at a time

- Each token depends on the last
- Single token: matrix $\times$ vector
- Bandwidth-bound: bytes per weight
- Drives tokens per second

prompt $\rightarrow$ **prefill** $\rightarrow$ first token $\rightarrow$ **decode** $\rightarrow$ decode $\rightarrow$... $\rightarrow$ stop

Quantized weights stay quantized

On the GPU, too. Nine formats, each with its own kernel class; unpacking is a loading decision.

FLOAT

F32

F16

BF16

8-BIT

Q8_0

BLOCK-QUANTIZED

Q4_0

Q4_1

Q4_K

Q5_K

Q6_K

Fused dequantization in the matvec: +20–30 % (PR #78).
Unsupported formats are refused, never silently converted.

LLAMA-3.2-1B, SAME FILE, RETAINED AS

Q4_0

172.4 tok/s

decode

1570 MiB

GPU memory peak

Q8_0

136.0 tok/s

decode

2060 MiB

GPU memory peak

The decode optimization ladder

All in the tree today, all written in Java.

01

39 %

Fused kernels

RMSNorm + gate/up: 39 % of GPU time, the top kernel

02

Warp-per-row matvec

butterfly shuffles via WARP_SHUFFLE; SIMD32 on Metal

03

Split-KV flash attention

online softmax, lane-cooperative decode

04

FP16 KV cache

default; $\cos \geq 0.99999$, top-1 100 %

05

Paged KV

one block pool, block table walked in-kernel

06

5.4×

CUDA graphs

capture once, replay per token

07

~500×

On-device sampling

greedy argmax on the GPU

08

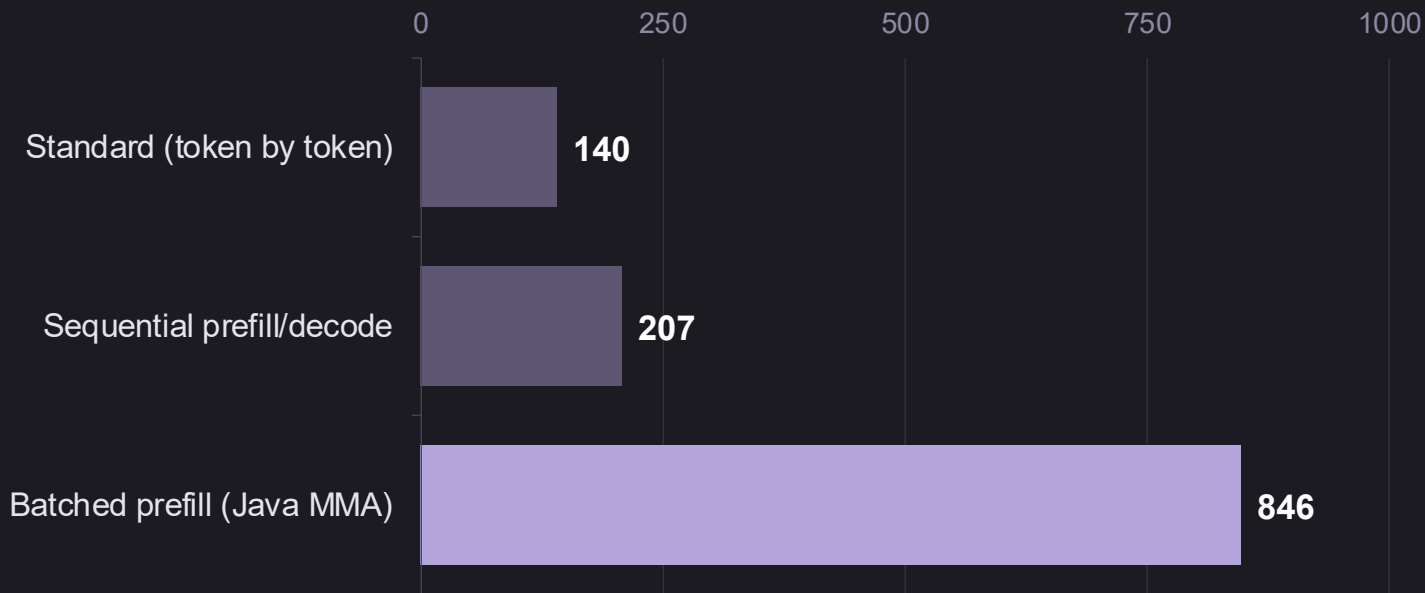
41×

Static batched decode

B sequences per step, bit-exact (PR #129)

Prefill is compute: use tensor cores

Qwen3-0.6B F16 prefill on CUDA · CI history, RTX 5070 Ti, 2026-09-21 · trend, not a benchmark



6×

batched vs standard prefill (140 → 846)

5.2×

Llama-3.2-1B F16: 156 → 810 tok/s

41×

aggregate, static batched decode, RTX 4090, bit-exact

Off by default: Q8_0 through FP16 MMA missed CPU parity (0.76 % vs 0.01 % budget).

A · A declarative Quarkus AI service

Describe the service, set the model. Quarkus wires the rest; the engine runs inside your JVM.

1 · CONFIGURE

```
# application.properties
quarkus.langchain4j.jitllm
  .chat-model.model-name=\
    unsloth/Llama-3.2-1B-Instruct-GGUF
  .chat-model.temperature=0.0
```

2 · DESCRIBE

```
@RegisterAiService
interface Explainer {
    @UserMessage("Explain {topic}.")
    String explain(String topic);
}
```

3 · INJECT

@Inject Explainer

call it like any bean
answers come from the local
GPU

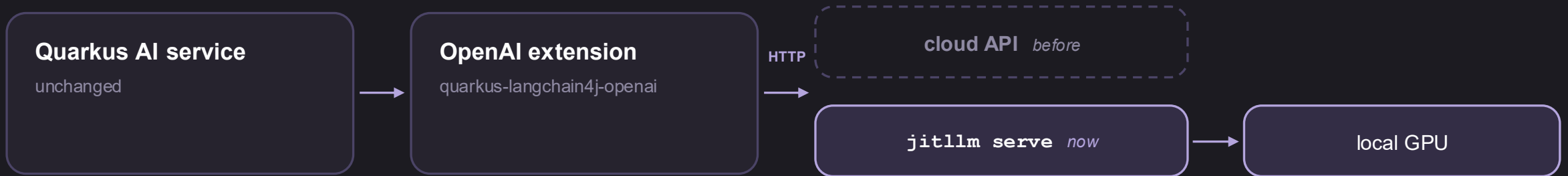
model downloaded from Hugging Face on first
use

engine loaded once, in process

GPU Java kernels via TornadoVM

C · Cloud to local GPU is a base-url change

Same AI service, the OpenAI extension, one line different. The data never leaves the machine.



THE ONLY CHANGE · application.properties

```
- quarkus.langchain4j.openai.base-url=https://api.openai.com/v1
+ quarkus.langchain4j.openai.base-url=http://localhost:8089/v1
```

same answer every run

streaming works

/v1/models reports context length

wrong model name: a clear 400

```
./jittllm serve -m model.gguf --gpu -c 4096 --port 8089
```

Spring Boot: the same trick

Two routes: point Spring AI at jitllm serve, or load the model as a bean.

ROUTE A · CONFIGURATION ONLY

Spring AI, OpenAI starter

Spring app



OpenAI starter



jitllm serve

```
# application.properties
spring.ai.openai.base-url=
  http://localhost:8089
spring.ai.openai.api-key=local
```

no Java change: the controller is plain Spring AI

ROUTE B · ONE BEAN

The engine inside Spring

Spring app



LocalModel bean



GPU

```
@Bean(destroyMethod = "close")
LocalModel model() {
    return LocalModels.load(path,
        ModelOptions.defaults());
}
```

loaded once at startup, closed with the context

WHERE WE ARE TODAY

Pure Java, at llama.cpp speed on GPUs

Same GPU, same GGUF weights. Every kernel written in Java and compiled by TornadoVM at runtime.

0.94×

DECODE · 27B

of llama.cpp decode on Qwen3.8-27B

40.5 vs 43.2 tok/s · Q4_0 · pure JIT, zero native code

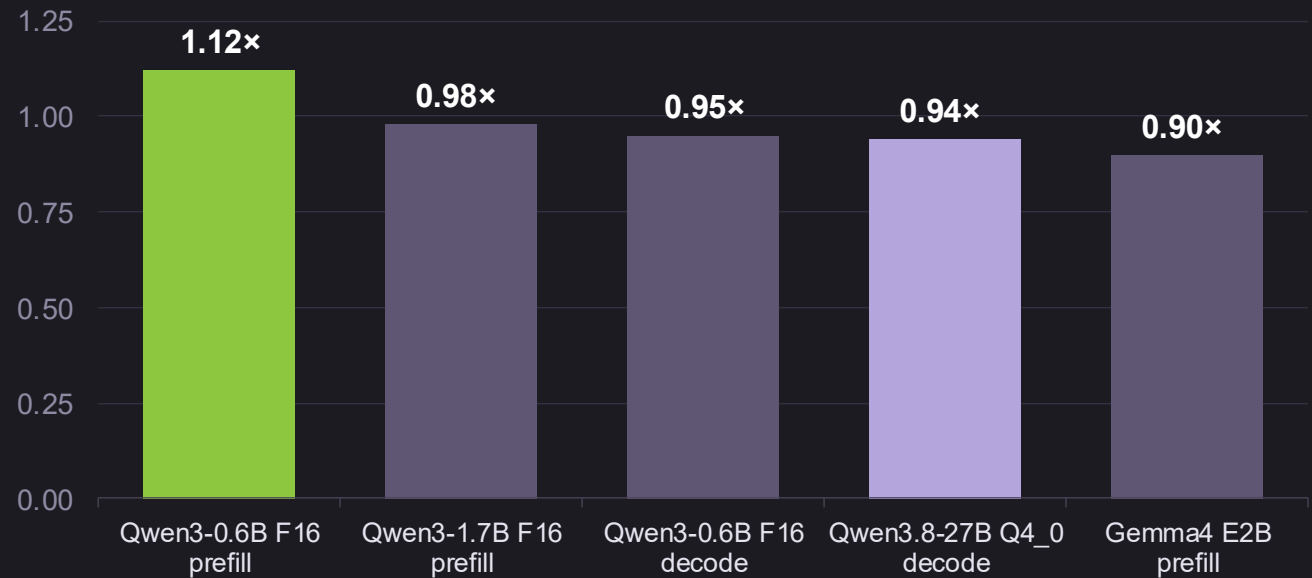
1.12×

PREFILL · HYBRID

faster than llama.cpp on prefill

Qwen3-0.6B F16 pp512 · 50,300 vs 44,805 tok/s · cuBLAS + Java

Throughput relative to llama.cpp = 1.00



5 of 16 measured configurations · llama.cpp at the same batch size · GPU: fill in

No C++, no CUDA C, no JNI.